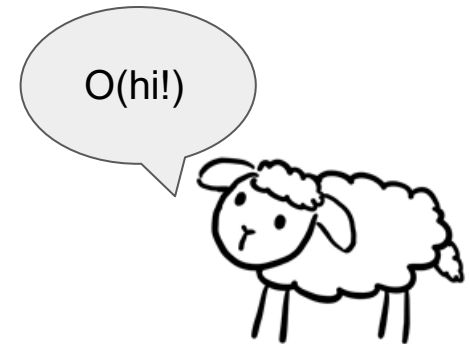
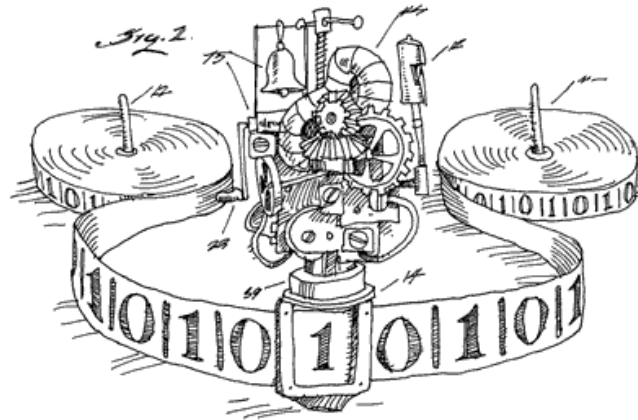


EECS 376: Foundations of Computer Science

Emily Graetz (rhymes with “pets”)
they/them



"Of all the courses I've ever taken, this was one of them"
- New York Times Review of CS Courses

"We cannot recommend this course too highly"
- The Michigan Daily



Sheep art by Melissa Zhang, mathematics professor at UC Davis

Happy Eat Outside Day

- Other Holidays:
 - National Trail Mix Day
 - National Diatomaceous Earth Day
 - National Matchmaker Day
 - Motorist Consideration Monday
 - Love Litigating Lawyers Day
 - We Love Memoirs Day
- This week is Be Kind to Humankind Week
- August is National Immunization Awareness Month

Course Staff

Instructors: 3



Nicole Wein,



Emily Graetz,



Mahdi Cheraghchi

Admin Support: 1 Rose Sherry

GSIs: 5

IAs: 4

Graders: ~10

You **can** attend:
- any lecture(s)
- any discussion(s)



Mascots: 1



Hi! My name is λ .
I'm a lamb, duh!

A Little About Me

- Call me *Prof. Graetz*, or *Emily*, or “hey”, ...
 - (“they” pronouns)
- **Brief academic bio:**
 - BS in CS/Math, Rose-Hulman Institute of Technology, 2013
 - MS in CS, MS in Math 2016, 2017
 - Professor at University of Michigan (EECS 203), 2016-pres.
- **Personal interests:** 1 kid, 2 cats, 1 dog, biking, video games
 - Sweet Adelines (women’s barbershop singing),
 - Board Games (e.g. Hanabi, Shipwreck Arcana, The Mind)
 - Puzzles (physical or [Conceptis](#), **not** jigsaw)

Learning Objectives

1. Given a challenging computational problem:
 - Determine whether or not a computer can solve it.
 - If *not*, prove it!
 - If so:
 - Design an algorithm for it
 - Derive its worst-case running time
 - Prove the correctness of the algorithm

Wait, some
problems are
unsolvable ?



Learning Objectives

2. Get comfortable with major techniques and paradigms:
 - Divide-and-conquer, greed, dynamic programming, the power of randomness

Problems that are **easy** for a computer
 - Computability

Problems that are **unsolvable** by a computer
 - NP-completeness and approximation algorithms

Problems that are **"probably hard"** for a computer
 - Cryptography

Using "probably hard" problems for our benefit: secure comms
3. Approach computational problems in a principled and rigorous way
 - In this course you will be asked to think in ways you have never thought before. *It may be extremely uncomfortable!*
 - This will make you a better programmer, problem solver, CS-ist.

Course Outline

- Algorithm Design & Analysis (7 lectures)
- Computability theory (6 lectures)
- Complexity theory (6 lectures)
- Randomized algorithms (3 lectures)
- Cryptography (3 lectures)
- Special topics (1 lectures)

Is this an EECS class?

- **Question:** Wolverine Access says it is an EECS class. Why does it feel like a math class?
- **Answer:** It's both!

The only way to answer the questions we raise is to **define** mathematical **models** and apply a **rigorous proof**-based methodology to the questions.

- **Question:** Why do we do proofs?

Answer: 1. To be confident in correctness. 2. To *understand*.

Why study CS foundations?

(a.k.a. theoretical computer science)



Top 7
Reasons

1. Theoretical ideas underlie a lot of software (e.g., hardware design, Internet routing, cryptography, Google, many more).

"Everyone knows Moore's Law — a prediction made in 1965 by Intel co-founder Gordon Moore that the density of transistors in integrated circuits would continue to double every 1 to 2 years...in many areas, performance gains due to improvements in **algorithms** have vastly exceeded even the dramatic performance gains due to increased processor speed." -Report to the President and Congress: Designing a Digital Future (2010)

2. It's very easy to write code that is either incorrect or will run for millions of years. Good theory can (often) fix this!

"For every complex problem there is an answer that is clear, simple, and wrong." -H. L. Mencken

Why study CS foundations?

(a.k.a. theoretical computer science)

A yellow starburst graphic with multiple points, containing the text 'Top 7 Reasons' in bold black font.

**Top 7
Reasons**

- 3. In your job you might be asked you to solve a problem that's provably impossible. Save yourself the trouble!
- 4. It will help you ace interviews.
- 5. Fundamentals apply to any computer: past, present, or future. Be future-proof!

Why study CS foundations?

(a.k.a. theoretical computer science)

Top 7
Reasons

6. Computation is everywhere (not just in computers)



Economics / Game Theory
e.g., auctions, matching markets



Collective animal behavior



Your brain

7. Inherent beauty, and connections to other areas of math

Struggling is good

This class asks you to comprehend deep ideas and solve challenging problems. You are expected to struggle!

It is important to have “wrong” ideas. The discoverers of every great idea in this course had many wrong ideas before having the right idea.

We are here to help! You will likely do better in this course if you ask questions during class, at office hours, and on Piazza.

An example of a good question to ask in class: "Can you repeat what you just said?"

The (AI)elephant in the room

What is the **point** of learning, designing, and analyzing algorithms anymore?

- AI is often **wrong** or **inadequate**: once you get to the frontier of the field, it cannot solve most problems
- You need **subject matter expertise** to **interact productively** with AI: ask the right questions and accurately interpret its output
- The real world is messy and you need to be able to **distill real-world problems** into well-defined computational problems

How to Succeed in 376 (“Cheat Codes”)

Attend & engage with:

- Lectures: key ideas, conceptual framework, and examples
- Discussions: reinforcement, practice problems
- Office Hours: detailed questions

Be able to reproduce proofs from lecture, course notes & HWs

Form a collaboration group and check each others' thinking

Have HWs “essentially done” by Tuesday; write up carefully

Ask & answer questions on Piazza (search first)

Administration

Website (eecs376.org) contains all resources:

- **Syllabus**
- **Schedule:** includes shared calendars for all events incl. office hours
- **Course Notes:** Follows lectures quite closely (but not exactly)
- **Files:** HWs, lecture slides, discussion worksheets, etc.
- **Lecture recordings**
- **Gradescope:** HW submission + HW/exam grades
- **Piazza:** questions and answers (feel free to answer!)
- **Forms:** admin form (arising issues, prolonged illness), exam conflict form
- **Emily's Office hours:**
 - Mon, Wed 1:30-2:30pm, in BBB 2733
 - I can often hang around after if you still have questions or want to ask a question one-on-one (e.g. about grades)

Administration

- 12 weekly HW assignments, due **Wednesdays 8pm**
 - Submissions open until 9:59pm, with 5% penalty
 - **No Late Submissions after 9:59pm**
 - Staff help only until 8pm
- Two lowest scores will be dropped
- Solutions published shortly after the deadline
- HW 1 will be posted by Wednesday evening, and is due next Wednesday at 8pm
- **Midterm:** Monday, October 26, 7-9pm
- **Final:** Wednesday, December 16, 7-9pm
- If you have an exam conflict: eecs376.org -> exam conflict form

Collaboration

We encourage collaboration! See syllabus for more details

- When writing your solution, the **write-up must be done individually and entirely on your own**, and **you may not look at anyone else's write-up, including your collaborators'**.
- When turning in work that benefited from a collaboration, **you must acknowledge your collaborators** (as your answer to question 0 on the assignment).

Encouraged Collaboration	Unacceptable Collaboration
Brainstorming solution ideas, e.g., whether to use a reduction for a problem and ideas about what a suitable language for the reduction might look like	Walking through an important piece of the solution step-by-step, sharing solution sketches or drafts, or otherwise directly giving away your solution to someone, whether written or verbally
Helping others understand the problem statement and nuances of the problem	Providing your solution as a reference

(see syllabus)



AI use is not permitted on the HW

Grading (see syllabus for details)

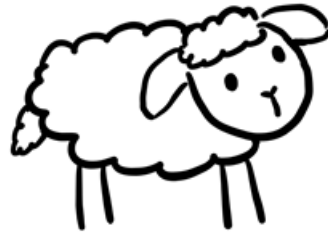
Lowest 2 HW scores dropped

Assignment	Weight
Homeworks	40%
Exams (midterm and final)	56-60%
Discussion attendance	0-3%
Course evaluations	0-1%
Total	100%

If you attend at least 10 out of the 14 discussions, you get the full 3%. Otherwise, your combined exam score will count as your score for this component. Same goes for the course evaluations 1%.

Additionally, you need at least a **45% exam average** to pass the course.

Now onto the course content...



Are ewe
ready?

Topic 1: Algorithms

Some historical events

Today + next time!

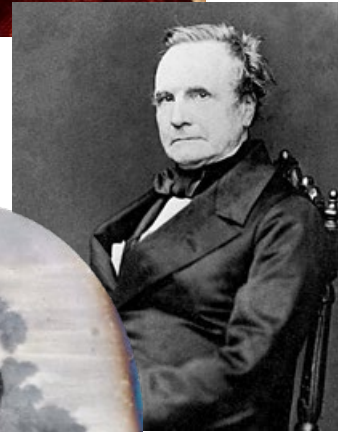
300BC: Euclid describes algorithms (for problems such as Greatest Common Divisor) that are still used today

825: Muhammad ibn Musa al-Khwarizmi writes “On the Calculation with Hindu Numerals.” The word “algorithm” will be named for al-Khwarizmi.

1837: Charles Babbage describes plans for Analytical Engine (mechanical general-purpose computer, never completed)

1842: In her notes on the Analytical Engine, Ada Lovelace writes the first algorithm specifically tailored for implementation on a computer (it calculated Bernoulli numbers)

(more history here: scottaaronson.blog/?p=524)



Review: Input Size

We express the running time of an algorithm as a function of its **input size**.

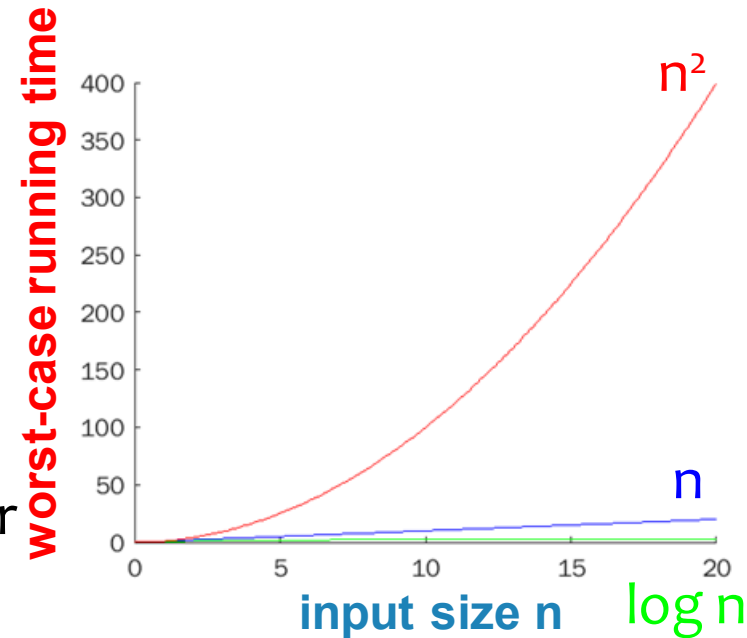
Common interpretations of input size (space to store it):

- size of array = # elements
- size of graph = # vertices + # edges
- size of integer = # **digits** (or # bits, doesn't matter which)
- **Rule of thumb:** size = # bits to represent input

Review: Running Time

We measure the efficiency of an algorithm by its **worst-case running time**: the maximum number of steps the algorithm can take on an **input of size n** .

Different computers running the same algorithm won't do exactly the same number of steps, but that's ok!



We abstract away such details with the help of **asymptotic notation**. We express running times in Big-O notation: e.g., $O(\log n)$, $O(n)$, $O(n^2)$ etc, where n is the **input size**.

"Efficient": running time polynomial in input size

Review: Asymptotic Notation

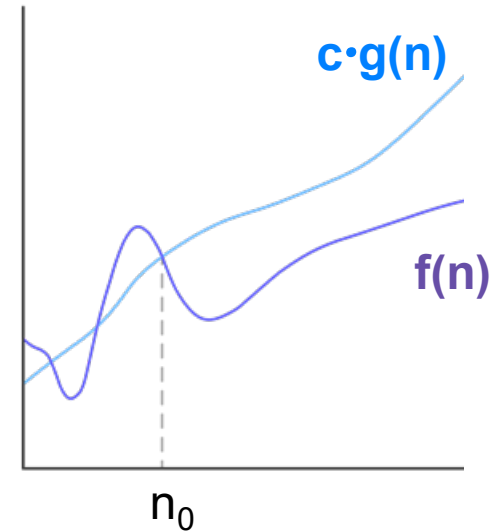
Formal definition of $f(n) = O(g(n))$:

there exist positive constants c, n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

$\geq$ for Ω instead of O

Informal definition of $f(n) = O(g(n))$: “at least” for Ω instead of O

$f(n)$ is asymptotically **at most** $g(n)$
(ignoring constant factors)



Which of these are $O(n^2)$?

$3n^2$ n^2+n $\log n$ $n^4 - n^3$ $9999 \cdot n$

$f(n) = \Theta(g(n))$ means $f(n)$ is both $O(g(n))$ and $\Omega(g(n))$.

Exponential vs. Polynomial



The λ -O-Matic performs 10^{11} operations/sec

	n=10	n=35	n=60	n=85
n^2				
n^3				
2^n				

"Efficient": running time polynomial in input size

Exponential vs. Polynomial



The λ -O-Matic performs 10^{11} operations/sec

	n=10	n=35	n=60	n=85
n^2	100 < 1 sec	1,225 < 1 sec	3,600 < 1 sec	7,225 < 1 sec
n^3				
2^n				

"Efficient": running time polynomial in input size

Exponential vs. Polynomial



The λ -O-Matic performs 10^{11} operations/sec

	n=10	n=35	n=60	n=85
n^2	100 < 1 sec	1,225 < 1 sec	3,600 < 1 sec	7,225 < 1 sec
n^3	1000 < 1 sec	43,000 < 1 sec	216,000 < 1 sec	614,000 < 1 sec
2^n				

"Efficient": running time polynomial in input size

Exponential vs. Polynomial



The λ -O-Matic performs 10^{11} operations/sec

	n=10	n=35	n=60	n=85
n^2	100 < 1 sec	1,225 < 1 sec	3,600 < 1 sec	7,225 < 1 sec
n^3	1000 < 1 sec	43,000 < 1 sec	216,000 < 1 sec	614,000 < 1 sec
2^n	1024 < 1 sec			

"Efficient": running time polynomial in input size

Exponential vs. Polynomial



The λ -O-Matic performs 10^{11} operations/sec

	n=10	n=35	n=60	n=85
n^2	100 < 1 sec	1,225 < 1 sec	3,600 < 1 sec	7,225 < 1 sec
n^3	1000 < 1 sec	43,000 < 1 sec	216,000 < 1 sec	614,000 < 1 sec
2^n	1024 < 1 sec	34,000,000,000 1/3 of a second		

"Efficient": running time polynomial in input size

Exponential vs. Polynomial



The λ -O-Matic performs 10^{11} operations/sec

	n=10	n=35	n=60	n=85
n^2	100 < 1 sec	1,225 < 1 sec	3,600 < 1 sec	7,225 < 1 sec
n^3	1000 < 1 sec	43,000 < 1 sec	216,000 < 1 sec	614,000 < 1 sec
2^n	1024 < 1 sec	34,000,000,000 1/3 of a second	1.15×10^{18} ~1 quintillion 4 months	

"Efficient": running time polynomial in input size

Exponential vs. Polynomial



The λ -O-Matic performs 10^{11} operations/sec

	n=10	n=35	n=60	n=85
n^2	100 < 1 sec	1,225 < 1 sec	3,600 < 1 sec	7,225 < 1 sec
n^3	1000 < 1 sec	43,000 < 1 sec	216,000 < 1 sec	614,000 < 1 sec
2^n	1024 < 1 sec	34,000,000,000 1/3 of a second	1.15×10^{18} ~1 quintillion 4 months	3.87×10^{25} ~39 septillion >12 Million Years

"Efficient": running time polynomial in input size

Exponential vs. Polynomial

An experiment to try with a child (or adult who hasn't taken 376):

If I pay you every day for a month, would you rather receive:

1. 1 penny on 1st day
2 pennies on 2nd day
4 pennies on 3rd day
8 pennies on 4th day
etc. (doubling each day)

OR

2. \$100 per day

"Efficient": running time polynomial in input size

Now for the oldest algorithm...

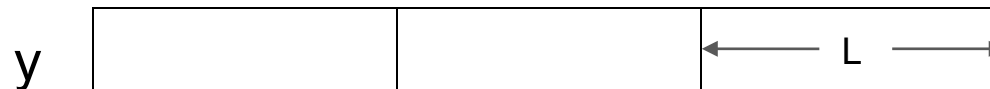
The Tiling problem (gcd: greatest common divisor)

Input: n-digit integers $x \geq y \geq 0$, not both = 0.

$$\log_{10} y < n$$
$$y < 10^n$$

Output: greatest common divisor $\text{gcd}(x,y)$
largest integer L that divides both x and y

In other words: largest integer tile length that can exactly tile paths of lengths x and y



I want to tile
two paths but
only buy one
size of tile



What is the simplest brute-force algorithm you can think of?

Algorithm:

Is this algorithm *efficient*?

What is the simplest brute-force algorithm you can think of?

Algorithm:

If $y = 0$ return x

For $i = y$ down to 1:

 If i divides x and i divides y , return i

Is this algorithm *efficient*? **(relative to input size $n = \text{digits}(x, y)$)**

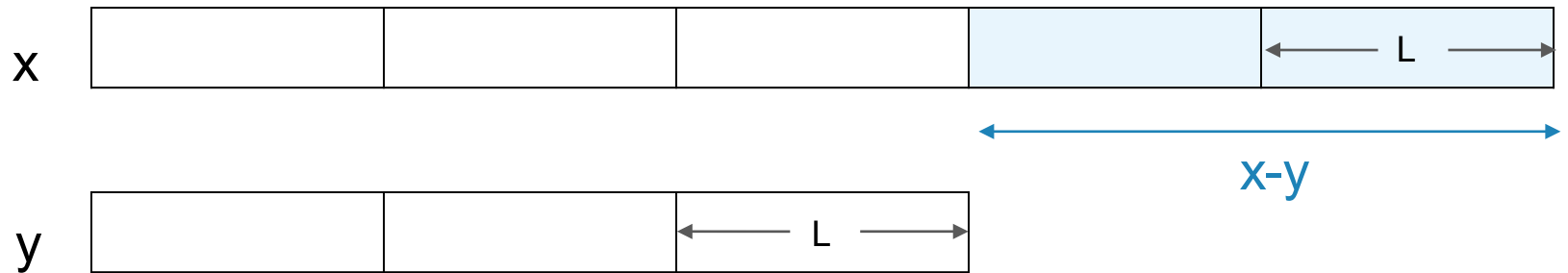
At most y iterations **Worst case, $y \approx 10^n$**

$O(y)$ divisions

$\sim O(10^n)$ divisions (NOT efficient)

Euclid's idea! (actually it was known before Euclid)

Claim: $\gcd(x, y) = \gcd(x-y, y)$.



Proof. We'll show that for any L : L divides x and $y \Leftrightarrow L$ divides $x-y$ and y .

1. ($\Rightarrow$) Suppose L divides both x and y .

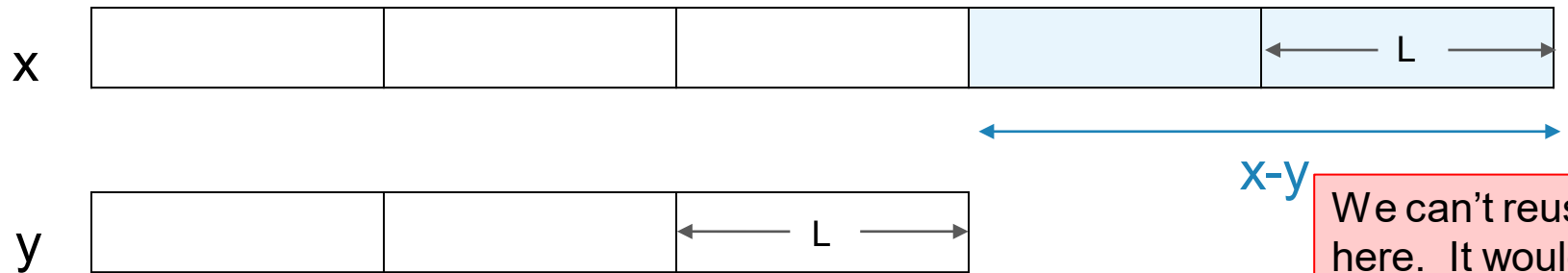
Why does this mean L divides $x-y$?

2. ($\Leftarrow$) Suppose L divides both $x-y$ and y .

Why does this mean L divides x ?

Euclid's idea! (actually it was known before Euclid)

Claim: $\gcd(x, y) = \gcd(x-y, y)$.



Proof. We'll show that for any L : L divides x and $y \Leftrightarrow L$ divides $x-y$ and y .

- $(\Rightarrow)$ Suppose L divides both x and y . ~~$x = kL$ and $y = kL$~~
 $x = k_1L$ and $y = k_2L$
Why does this mean L divides $x-y$?

Wait, why are we allowed to reuse k_1 here?

This is in the other case of the proof. We don't have access to the other k_1 .

L divides both $x-y$ and y .

L divides x ?

$$x - y = k_1L - k_2L = (k_1 - k_2)L$$

$$x - y = k_1L \text{ and } y = k_2L$$

$$x = (x - y) + y = k_1L + k_2L = (k_1 + k_2)L$$

Euclid's (more general) idea!

$$\gcd(x, y) = \gcd(x-y, y)$$

$$= \gcd(x-2y, y)$$

$$= \gcd(x-3y, y)$$

$$= \gcd(x-ky, y) \text{ for any integer } k$$

$$= \gcd(x \bmod y, y)$$

Now we have a smaller instance of the same problem.

How do we turn this *identity* into a *program* for computing gcd?

Euclid's *recursive* gcd algorithm

- $\text{Euclid}(x, y)$ // we'll assume that $x > y \geq 0$.

$\text{Return}(\text{Euclid}(x \bmod y, y))$

Euclid's *recursive* gcd algorithm

- $\text{Euclid}(x, y)$ // we'll assume that $x > y \geq 0$.

Base case:

If($y=0$) then return(x)

General case:

Return($\text{Euclid}(y, x \bmod y)$)

Correctness claim: $\text{Euclid}(x, y)$ returns $\text{gcd}(x, y)$.

Is this a “good” (polynomial time) algorithm?

Next time: a method for analyzing this type of algorithm.